

A CONTROL LAYER ON THE WAY OUT TO AI

AegIs

A local gateway between the employee (or an agent) and the model. The prompt does not leave the company until it has passed inspection.

What this deck covers

01 The problem: AI at work and data that already leaks

03 What AegIs is and where it sits in the traffic

05 Categories, confidence, roles and people

07 The dashboard, policy changes with no restart

02 Why DLP, proxies and training are not enough

04 How a decision is made: allow / redact / block

06 The incident: who, what, when, why

08 How to run it, and what we refuse to do

People paste their work into chat.

38%

at work, into chat

Companies have let ChatGPT, Copilot and agents into the working day. The employee is not “attacking the system”. They are in a hurry: review this commit, summarise this contract, draft a mail to a client.

The message box gets whatever they are sitting on — code, an HR sheet, a CRM export, a folder with `.env`. The model is outside the firm. Once the prompt leaves, you cannot pull it back. The vendor sees it, logs it, sometimes trains on it, sometimes surfaces it in another thread.

This is not a theoretical GDPR risk. It is a habit. AegIs assumes you will not train the habit away — you have to stop the packet *before* it leaves the machine.

One pasted folder.

Jake on frontend asks ChatGPT to review a commit before he pushes. The model says “paste the diff or drop the files”. Jake drops the whole directory. He does not want to redact.

- .env
- SSN_Cole.txt
- payroll_march.xlsx

SECRET

Keys in .env

Database token, Stripe, Slack. Once pasted, it lives in chat history. Rotation is already incident response.

PERSON

Identifiers

National ID, tax number, email, phone. Under GDPR that is disclosing personal data to a party outside the processing agreement.

COMPANY

Payroll

A salary sheet is not “context for a code review”. It is an HR document sitting in someone else’s model.

Why the tools you already bought miss this

DLP on mail and USB

It sees an Outlook attachment and a copy to a stick. It does not see the composer on chatgpt.com, claude.ai or an agent window. The traffic is HTTPS from the user’s browser, often a personal account.

A proxy without a body

A plain HTTP proxy on HTTPS sees the host (SNI), not the prompt. Reading the body means a corporate CA and MITM — and you still will not break pinning on consumer chats. That is the wrong fight.

Training and a policy PDF

People know the ban. Then they have a deadline. One “here’s the whole folder” walks around a rule nobody enforces at the moment of Enter.

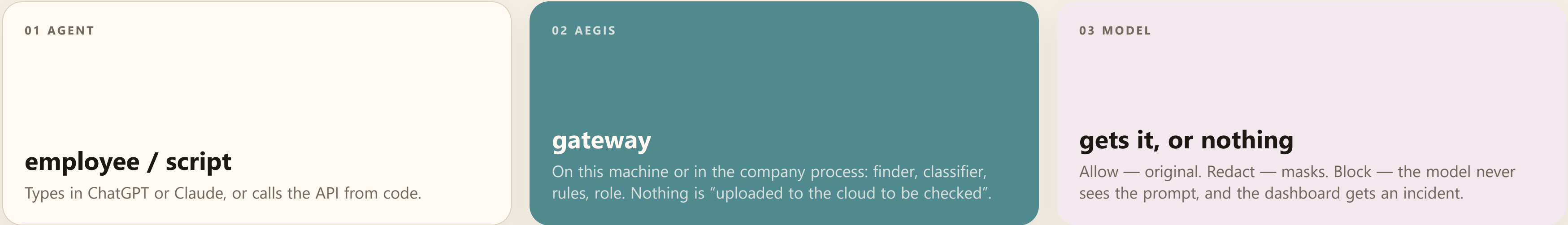
Agents outside the browser

SDKs, cron, LangChain, a CI script call the API directly. There is no extension there. They need a gateway on `OPENAI_BASE_URL` or a company proxy — the same decision engine.

DEFINITION

AegIs is a gateway, not another chatbot.

It sits between whoever writes (a person or a program) and the model. Every prompt is checked locally. Only the verdict decides whether anything goes out.



ChatGPT, Copilot and the agent stay. We do not move people into a "safe company chat". They keep their tools. The leak does not.

Three doors, one engine.

Browser, SDK or a background job — the verdict, the policy and the log are the same.

BROWSER

Chrome extension

Reads the composer on ChatGPT, Claude, Gemini, Copilot, Perplexity, Grok, DeepSeek and others. Before Enter / Send it calls `POST /v1/scan`. It can stop the send and show what it blocked.

AGENTS / SDK

API gateway

The app sets `OPENAI_BASE_URL` to `http://127.0.0.1:8080/net/openai/v1`. Same idea for Anthropic and Ollama. Helios scans the prompt JSON, then — if allowed — forwards upstream.

PROCESSES

HTTP proxy :8888

For tools that honour `HTTP_PROXY`. On HTTP you see the body. On HTTPS without a CA — the host. That is why AI agents should use `BASE_URL`, not “MITM the world”.

Three ways out of the gate

ALLOW

Pass

No sensitive spans, or the role / whitelist says this person may send this. The model gets the original. The audit line still exists.

REDACT

Redact

IDs, email, card, phone become masks ([PESEL] , [EMAIL]). The rest of the prompt goes. The employee sees what was cut and can fix it and try again.

BLOCK

Stop

Secrets, critical identifiers, jailbreak. Stop in the UI, 403 on the API. The model gets nothing. The dashboard gets who, what, when, which control.

Default preset is **balanced**: government_id , financial , credentials and attack block; health redacts. An admin changes that with a slider, not a deploy.

How we know it is an ID number

Finder	Pulls candidate spans with the same regex families as the Laya pii training set, plus PESEL, NIP, REGON, IBAN, API keys, tokens, passwords.
Laya · pii	An open decision classifier (Apache-2.0 weights). 22 labels: given name, phone, card, passport... It gets a span and a closed answer list, and returns calibrated probabilities. It does not generate text and does not send content to a closed API.
Hard rules	A valid PESEL / NIP, card Luhn, IBAN and secrets stay even when the classifier says not pii . Checksums are not “the model’s opinion”.
Signatures	Jailbreak, prompt injection, exfiltration, tool abuse — an OWASP-LLM-style catalogue, CERT import, toggled per control.
Heuristic	Without downloading weights (~850 MB) the gate still boots: backend=heuristic . Semantic controls show as off; the rest works. On a judge’s laptop that is the same product.

Admin decides what counts — and how sure.

A category is not a frozen list from a policy PDF. It is a label with a risk, an action and a confidence floor. Laya scores the span; below the slider, the category does not fire.

CATEGORY	EXAMPLES	DEFAULT ACTION
government_id	national ID, tax number, passport	block
credentials	API key, token, password, private key	block
financial	card (Luhn), IBAN	block
attack	jailbreak, injection, exfiltration	block
health	medical history, referrals	redact
contact / person_name	email, phone, name	depends on destination and role

Custom categories are added in the panel. Presets `strict` | `balanced` | `permissive` . Per-destination overrides: localhost is not chatgpt.com. `policy.json` reloads on mtime — no process restart.

A role is an exception, not a job title.

A developer, a doctor and an accountant leak different things. One global block either annoys everyone or catches nothing. A role says: *this person may send these categories*.

ROLE	EXCEPTIONS	WHY
Employee	none	Any PII excursion lands with a manager. Default for a new person.
HR	name, contact, demographic	Recruiting has to paste a candidate’s name and phone. National ID still no.
Finance	financial	A vendor IBAN or a test card is not an incident. An API key still is.
Manager	everything + override	Sees the dashboard, can override a block when it is genuinely needed.

The same category can block a developer and allow an accountant. Roles are edited in the panel; exceptions are category checkboxes, not a new deploy.

Assignment applies on the next message.

People page

Each person has an email, a team and a `role_id`. The extension and the gateway know who is typing (account / `X-Employee-Id`). The verdict is computed against *that* role, not “everyone in the company”.

Unknown sender

Someone arrives from an account that is not on the list. They land in the “unknown” queue — they do not vanish in the logs and they do not inherit a manager’s silent exceptions.

No restart

Move Magda from Employee to HR. You do not reload the extension, restart the gate, or rebuild a Docker image. The next Enter already uses the new policy. The banner in the panel shows the new version.

Whitelist

Separate no-alert exceptions: a test address, a company account number, a phrase that is a false positive. The whitelist does not switch the rest of the gate off.

A block with no incident is theatre.

When Aegls stops a prompt, the dashboard gets a record. Not “some alert”. Something you can read and act on.

What is in the record

- who — person, email, team, role
- where — chatgpt.com, OpenAI API, agent, proxy
- what — categories and labels, a masked snippet
- why — which controls fired, confidence, action
- when — timestamp and gate latency

What a manager does with it

- a queue with filters, grouped repeats (5-minute dedupe)
- bulk actions, assignment, a note, a runbook
- CSV / JSONL export for audit
- values stay masked; `reveal` itself is written to the log

This is a trail for GDPR and for a conversation with a human (“you pasted payroll”), not a dump of the whole chat. The content never reached the model — and it does not have to reach a cloud SIEM just to exist.

Anything a judge can change is visible immediately.

MONITOR

Posture, incidents, audit

A 0–100 score, “today” cards, 24h bars, top people and destinations. The incident queue. An append-only log. Telemetry: p50/p95, RPM, backend split, upstream errors. “Check a prompt” hits the same `/v1/scan`.

GOVERN

Controls and policy

Toggle checksums, the classifier, signatures, roles, whitelist. A threshold slider. Action per category and per destination. Presets. History of `policy.json`. Roles, categories, people — edited in place.

LIVE

Next request, not a restart

Change the threshold, disable a control or assign a role and a “Policy reloaded (version N)” banner appears. `GET /health` returns the new version. Demo chat walks the full path through `/net/openai`.

DEGRADATION

It still boots without Laya

No weights = heuristic + a banner. Deterministic PESEL / NIP / IBAN / Luhn / secrets / signatures stay. Semantics go grey. The UI does not pretend a model is watching.

Stop on the machine. In ~28 ms.

Classification is not a round-trip to the model vendor. The finder and hard rules are cheap. Laya, if enabled, runs locally. Order of magnitude: tens of milliseconds, not seconds of “please wait, we are scanning in the cloud”.

That matters on Enter in a chat and for an agent that is already waiting on tokens. The gate cannot be a second model.

If the verdict is block, the packet dies here. There is no “send it and do incident response later”.

ORDER

One request

model allow-list → rate limit → budget → loop guard → finder + classifier →
signatures → patterns → disabled controls → whitelist / role → per-category action
→ redaction → incident → audit + telemetry.

What we deliberately do not do

We do not break consumer chats

No MITM on Messenger, WhatsApp, pinned Cursor TLS. There is no `OPENAI_BASE_URL` there. For AI chats in the browser there is an extension that reads the field, not the whole TLS session.

We are not DLP for everything

We do not replace antivirus, disk encryption or mail DLP. We are a layer on the *way out to a model*. The rest of the stack stays.

We do not pretend the cloud classifies better

Sending the prompt to a third-party API “to check if it is sensitive” is the same leak. AegIs has to work without that step.

We do not hide degradation

Without Laya you see heuristic. Without an upstream key the gate still blocks first — a vendor 502 does not hide our 403.

After clone — one command.

On PATH: Python 3.10+ and Node 18+. The script does the rest: venv, dependencies, dashboard build, gate and proxy.

INSTALL

`python start.py`

Windows: `py start.py` or `.\start.ps1` . macOS / Linux: `./start.sh` . Heuristic by default (fast start). `--with-laya` pulls the model.

ADDRESSES

What comes up

Dashboard `http://127.0.0.1:8080/app` · demo chat `/demo` · API `/v1/scan` · gateway `/net/openai/v1` · proxy `:8888` .

Extension: Chrome → `chrome://extensions` → developer mode → Load unpacked → the `extension/` folder. Then ChatGPT / Claude / Gemini — a marker in the corner, and Enter on a national ID does not leave.

AegIs

A local gateway. The prompt does not leave until it has passed inspection. Policy per category and per role. The incident says who and what — before the model ever sees a thing.